

SQL Data Analysis

Project 3 — Querying for Truth | DecodeLabs Batch 2026

Dataset: orders (SQLite) — loaded from Cleaned_Dataset_Project1.xlsx, 1,200 records, 14 fields

Engine: SQLite 3 | **Tool:** Python (sqlite3, pandas)

Analyst: Intern, Data Analytics Track

Scope: 15 SQL queries covering SELECT, WHERE, ORDER BY, GROUP BY, HAVING, subqueries and CASE logic

1. Problem statement

Project 2 identified several business signals in the order data using Python. Project 3 re-derives and extends those findings using pure SQL — the language of relational databases and the standard tool for extracting answers directly from production data stores. The goal is to demonstrate that structured queries alone (filtering, grouping, and aggregating) can turn 1,200 raw order rows into specific, defensible business answers.

2. Methodology

The cleaned dataset was loaded into a SQLite table named **orders**. Every query below was written and executed against that table — these are not simulated results. Queries are grouped into three tiers of complexity: **(1) Filtering & Sorting** — SELECT, WHERE, ORDER BY; **(2) Aggregation Fundamentals** — GROUP BY with COUNT/SUM/AVG and HAVING; **(3) Advanced Analysis** — multi-column grouping, subqueries, and CASE-based bucketing.

A key engineering principle guided every query: the database does not execute SQL in the order it is typed. The actual **logical execution order** is FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY. This is why HAVING (not WHERE) must be used to filter on an aggregated value, and why a column alias defined in SELECT cannot be reused in that same query's WHERE clause.

3. Tier 1 — Filtering & Sorting

3.1 Q2. WHERE — comparison filter

Find all high-value orders (TotalPrice > \$2,500)

```
SELECT OrderID, Product, Quantity, UnitPrice, TotalPrice
FROM orders
WHERE TotalPrice > 2500
ORDER BY TotalPrice DESC;
```

OrderID	Product	Quantity	UnitPrice	TotalPrice
ORD200789	Tablet	5	691.28	3,456.40
ORD201122	Monitor	5	678.19	3,390.95
ORD200632	Laptop	5	678.16	3,390.80
ORD200469	Chair	5	676.98	3,384.90
ORD200328	Tablet	5	674.04	3,370.20
ORD200107	Printer	5	670.75	3,353.75
ORD200326	Laptop	5	670.48	3,352.40
ORD201065	Printer	5	666.80	3,334.00

Showing 8 of 94 rows.

94 of 1,200 orders (7.8%) exceed \$2,500 — this is the high-value order segment worth flagging for priority fulfillment and fraud review given its outsized revenue contribution.

3.2 Q4. WHERE — multiple conditions (AND / OR)

Find Cancelled or Returned orders paid by Credit Card

```
SELECT OrderID, PaymentMethod, OrderStatus, TotalPrice
FROM orders
WHERE (OrderStatus = 'Cancelled' OR OrderStatus = 'Returned')
AND PaymentMethod = 'Credit Card'
ORDER BY TotalPrice DESC;
```

OrderID	PaymentMethod	OrderStatus	TotalPrice
ORD200527	Credit Card	Cancelled	3,267.35
ORD200889	Credit Card	Cancelled	3,253.60
ORD200967	Credit Card	Returned	3,194.60
ORD200764	Credit Card	Cancelled	3,137.15
ORD200010	Credit Card	Returned	3,129.85
ORD200731	Credit Card	Returned	2,922.30
ORD200002	Credit Card	Cancelled	2,753.40
ORD200523	Credit Card	Returned	2,631.16

Showing 8 of 103 rows.

103 orders combine Credit Card payment with a Cancelled or Returned status. Cross-referencing payment method against fulfillment outcome like this is the first step toward identifying whether a specific payment channel correlates with higher failure rates.

3.3 Q5. ORDER BY — top 10 highest-value orders

```
SELECT OrderID, CustomerID, Product, TotalPrice
FROM orders
ORDER BY TotalPrice DESC
LIMIT 10;
```

OrderID	CustomerID	Product	TotalPrice
ORD200789	C57276	Tablet	3,456.40
ORD201122	C38840	Monitor	3,390.95
ORD200632	C67260	Laptop	3,390.80
ORD200469	C13877	Chair	3,384.90
ORD200328	C18404	Tablet	3,370.20
ORD200107	C16775	Printer	3,353.75
ORD200326	C65986	Laptop	3,352.40
ORD201065	C47778	Printer	3,334.00

Showing 8 of 10 rows.

The single largest order in the dataset is \$3,456.40 (Tablet, Qty 5) — consistent with the outlier analysis from Project 2, confirming these are legitimate maximum-basket purchases, not data errors.

4. Tier 2 — Aggregation Fundamentals

4.1 Q7. GROUP BY + SUM + AVG — revenue performance per product

```
SELECT
Product,
COUNT(*) AS OrderCount,
SUM(TotalPrice) AS TotalRevenue,
ROUND(AVG(TotalPrice), 2) AS AvgOrderValue
FROM orders
GROUP BY Product
ORDER BY TotalRevenue DESC;
```

Product	OrderCount	TotalRevenue	AvgOrderValue
Chair	178	195,620.11	1,098.99
Printer	181	195,612.61	1,080.73
Laptop	173	192,126.56	1,110.56
Tablet	179	186,568.95	1,042.28
Monitor	163	175,651.41	1,077.62
Desk	170	167,459.93	985.06
Phone	156	151,722.39	972.58

Chair (\$195,620) and **Printer (\$195,613)** are effectively tied for top revenue, while **Phone (\$151,722)** trails despite a competitive order count (156) — its lower unit price, not weak demand, explains the gap. This matches the Project 2 finding exactly, now derived independently via SQL.

4.2 Q8. GROUP BY + HAVING — filter aggregated groups

Payment methods with average order value above \$1,050

```
SELECT
PaymentMethod,
COUNT(*) AS OrderCount,
ROUND(AVG(TotalPrice), 2) AS AvgOrderValue
FROM orders
GROUP BY PaymentMethod
HAVING AVG(TotalPrice) > 1050
ORDER BY AvgOrderValue DESC;
```

PaymentMethod	OrderCount	AvgOrderValue
Credit Card	234	1,127.55
Gift Card	230	1,070.97
Cash	246	1,056.04

HAVING filters on the *aggregated* AVG(TotalPrice), which is only possible after GROUP BY has run — a plain WHERE clause cannot reference an aggregate. **Credit Card (\$1,127.55 avg)** is the strongest-spending payment channel.

4.3 Q9. GROUP BY on OrderStatus — the "cancel/return risk" query

```
SELECT
OrderStatus,
COUNT(*) AS OrderCount,
SUM(TotalPrice) AS Revenue,
ROUND(100.0 * COUNT(*) / (SELECT COUNT(*) FROM orders), 1) AS PctOfOrders
FROM orders
GROUP BY OrderStatus
ORDER BY OrderCount DESC;
```

OrderStatus	OrderCount	Revenue	PctOfOrders
Cancelled	250	276,396.21	20.80
Returned	247	243,277.70	20.60
Pending	237	256,328.15	19.80
Shipped	235	246,159.58	19.60
Delivered	231	242,600.32	19.30

The most important finding in this project: Cancelled (20.8%) and Returned (20.6%) orders together account for **41.4% of all orders** and **\$519,673.91 in revenue** — independently confirming the Project 2 EDA result via raw SQL aggregation. This is not a data artifact; it reproduces identically across both analysis methods.

5. Tier 3 — Advanced Analysis

5.1 Q12. CASE WHEN — bucket orders into value tiers, then aggregate

```
SELECT
CASE
WHEN TotalPrice < 500 THEN '1. Under $500'
WHEN TotalPrice < 1000 THEN '2. $500 - $999'
WHEN TotalPrice < 2000 THEN '3. $1,000 - $1,999'
ELSE '4. $2,000+'
END AS ValueTier,
COUNT(*) AS OrderCount,
SUM(TotalPrice) AS Revenue
FROM orders
GROUP BY ValueTier
ORDER BY ValueTier;
```

ValueTier	OrderCount	Revenue
1. Under \$500	383	101,696.90
2. \$500 - \$999	305	219,504.99
3. \$1,000 - \$1,999	332	482,242.95
4. \$2,000+	180	461,317.12

Orders in the **\$1,000–\$1,999** and **\$2,000+** tiers make up only 512 of 1,200 orders (43%) but generate **\$943,560** — **61% of total revenue**. Revenue is concentrated in a minority of larger baskets, reinforcing that retention/fulfillment fixes targeting this segment carry the most financial weight.

5.2 Q13. Referral source performance — marketing channel ROI signal

```
SELECT
ReferralSource,
COUNT(*) AS OrderCount,
ROUND(AVG(TotalPrice), 2) AS AvgOrderValue,
SUM(TotalPrice) AS TotalRevenue
FROM orders
GROUP BY ReferralSource
ORDER BY TotalRevenue DESC;
```

ReferralSource	OrderCount	AvgOrderValue	TotalRevenue
Instagram	259	1,062.88	275,285.45
Email	250	1,047.23	261,808.55
Google	241	1,039.18	250,441.48
Facebook	228	1,098.29	250,410.90
Referral	222	1,021.69	226,815.58

Instagram is the top-performing referral channel by both volume (259 orders) and total revenue (\$275,285), narrowly ahead of Email. **Facebook** has the highest average order value (\$1,098.29) despite fewer orders — a smaller but higher-intent audience.

5.3 Q14. Coupon impact — does a coupon change basket size?

```
SELECT
CouponCode,
COUNT(*) AS OrderCount,
ROUND(AVG(ItemsInCart), 2) AS AvgItemsInCart,
ROUND(AVG(TotalPrice), 2) AS AvgOrderValue
FROM orders
GROUP BY CouponCode
ORDER BY AvgOrderValue DESC;
```

CouponCode	OrderCount	AvgItemsInCart	AvgOrderValue
FREESHIP	313	5.53	1,070.41
SAVE10	286	5.46	1,065.87
NONE	309	5.56	1,043.37
WINTER15	292	5.38	1,035.90

Coupon usage shows only marginal differences in order value (a \$34 spread between FREESHIP and WINTER15) — discounting does not appear to meaningfully change basket size or spend in this dataset, suggesting coupons are not the growth lever to focus on.

5.4 Q15. Top 5 customers by lifetime spend (repeat customer check)

```
SELECT
CustomerID,
COUNT(*) AS NumberOfOrders,
SUM(TotalPrice) AS LifetimeSpend
FROM orders
GROUP BY CustomerID
HAVING COUNT(*) > 1
ORDER BY LifetimeSpend DESC
LIMIT 5;
```

CustomerID	NumberOfOrders	LifetimeSpend
C38840	2	5,723.23
C97593	2	2,855.22
C98474	2	2,175.34
C70659	2	1,853.96
C94569	2	1,675.35

Only 11 of 1,189 unique customers (0.9%) placed more than one order. This is a low repeat-purchase signal sitting alongside the 41.4% cancel/return rate — together they suggest an acquisition-heavy, retention-weak business model. Fixing fulfillment issues could compound: fewer cancellations may also improve the near-zero repeat rate.

6. Recommendations

- **Prioritize the 41.4% cancel/return rate** — confirmed independently by both Python (Project 2) and SQL (Project 3). This is the highest-confidence finding in the entire analysis.
- **Investigate the Credit Card + Cancelled/Returned overlap (103 orders)** as a possible starting point — check whether a specific payment flow is linked to higher failure rates.
- **Focus retention efforts on the \$1,000+ order tier**, which drives 61% of revenue from 43% of orders.
- **Treat the 0.9% repeat-purchase rate as a second core problem**, not a side note — a business this acquisition-dependent is vulnerable to rising customer acquisition costs.
- **Deprioritize coupon strategy changes** — the data shows minimal impact on order value; the ROI is likely higher elsewhere.

7. Appendix — full query catalog

All 15 queries are provided in the accompanying file **project3_queries.sql**, runnable against any SQLite database loaded with the same schema. Queries not detailed above (Q1, Q3, Q6, Q10, Q11) cover basic SELECT, LIKE pattern matching, product order-volume counts, multi-column grouping, and above-average subquery filtering respectively — included in the script for completeness and reuse.

Verified insight — SQL and Python analyses agree on every core finding. Ready for Project 4.